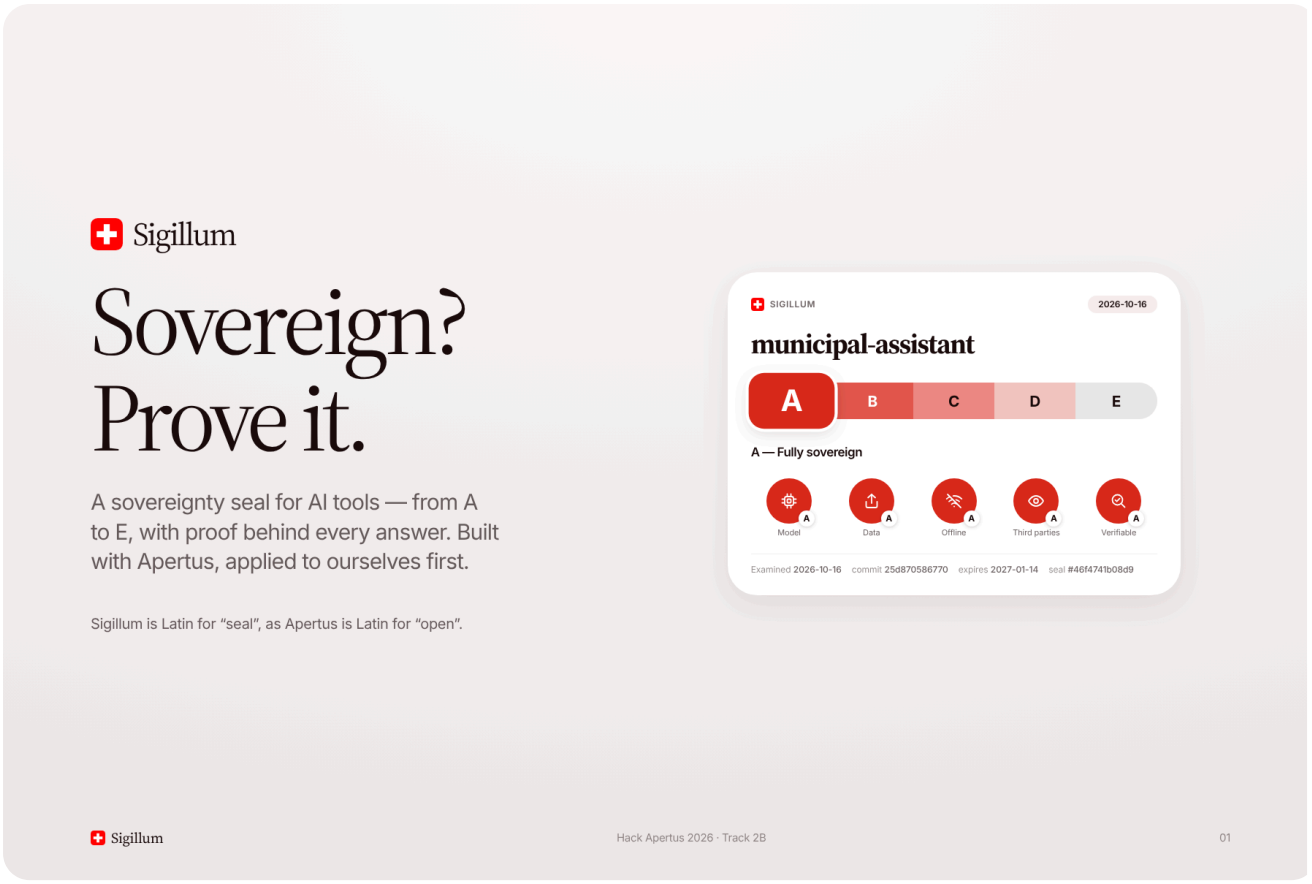


Sigillum — Devpost submission

Project name (title)

Sigillum — the sovereignty seal for AI

Tagline / subtitle (elevator pitch)

Everyone says "sovereign AI". Nobody measures it. Sigillum grades any AI project from A to E — with proof behind every answer — and graded itself first.

Sponsor / special prize to select

Hack Apertus Online — Winner Apertus Adoption: Own project (Track 2B)

Built with — 25 tags

apertus · apertus-v1.5 · swiss-ai · llama.cpp · gguf · python · python-standard-library · javascript · html · css · docker · docker-compose · json-schema · structured-outputs · hugging-face · static-analysis · sovereign-ai · digital-sovereignty · ai-transparency · privacy · air-gapped · on-premise · open-source · multilingual · romansh

"Try it out" links

- Live demo (static, real reports): https://sigillum-seal.netlify.app

- Git repository (public): `<YOUR-REPO-URL>` — project root `track_2b/`, `make run`
- Demo video (≤ 2 min): `<YOUR-VIDEO-URL>` (file: `track_2b/docs/video/sigillum-demo.mp4`)

About the project (Devpost story)

Inspiration

"Sovereign" is everywhere in European AI — on landing pages, in tenders, in hackathon pitches. And it means nothing precise. A tool can run on an open model and still send every question to a foreign API, embed an analytics script, or stop the moment the connection drops. A public buyer cannot tell a sovereign tool from one that only says so, and two teams who say "sovereign" do not mean the same thing.

We have seen this movie before. People bought fridges on the seller's word until the energy label arrived; they read ingredient lists nobody understood until the Nutri-Score. What changed behaviour was not a law alone — it was a simple sign backed by a public method. Apertus was born to give Switzerland an open, trustworthy foundation. The more its ecosystem grows, the more it needs a shared way to say what is *really* sovereign — before the word wears out. *Sigillum* is Latin for "seal", as *Apertus* is Latin for "open".

What it does

Sigillum asks any AI project five questions and looks for a verifiable trace — a file and a line anyone can open — behind every answer:

1. **Where does the model live?** On hardware the organisation controls, in Switzerland, or at a foreign provider?
2. **Where does the data go?** Nothing leaves, it stays in Switzerland, or it leaves the country?
3. **What happens without internet?** Everything works (tested), degraded, or everything stops?
4. **Who else is in the loop?** Third-party services, telemetry on by default, components nobody can audit?
5. **Can it be verified?** Open licence, reproducible build, tests — or nothing?

The result is one letter from **A** (fully sovereign) to **E** (dependent on foreign services) and five pictograms, readable by a mayor, a CIO or a pupil in ten seconds. Three rules make it credible:

- **The proof rule** — a claim without a trace never earns more than C. When information is missing, the seal says **unknown** instead of guessing, and unknown caps the grade at C.
- **The weakest link** — the final grade is the worst of the five answers. Perfect on four questions but data goes abroad? Not an A.
- **A dated seal** — date, commit, method version, SHA-256 fingerprint; it expires after 90 days.

Apertus reads, reconciles and explains — the evidence decides. Apertus extracts what a project's documentation claims about itself, sets each claim against the traces found in the code and names the gap ("the project says *everything runs locally, no data leaves the school*; the traces show usage statistics sent to Streamlit in the US"), and writes the verdict in plain German, French, Italian and Romansh (with the 8B model on a CPU, its lines mostly confirm our reviewed templates — and each report says so). Every Apertus answer is checked: quotes must exist word for word, evidence IDs must exist, a line in the wrong language is rejected — and rejections are counted in the report. The grade itself never depends on the model.

Each report also lists the concrete fixes to move up a letter, keeps the before/after history, and provides an embeddable badge. A team we simulated went from **D to B** with three fixes.

We applied it to the community — without naming anyone. Hack Apertus submissions are public only after 24 October, so we applied the seal to the 26 public Hugging Face Spaces that use Apertus, aggregated and anonymous: **none reaches A or B; 42 % call the model through a foreign service; 73 % ship telemetry on by default** (most often Gradio's usage analytics — a one-line fix).

And we started with ourselves. Sigillum's first examination of itself gave **D**: no licence file in the project folder. We fixed it, re-examined: **A**. Then we published our limits.

How we built it

- **Evidence collector** (Python standard library only): walks a repository, separates runtime code from build files and documentation, and records network destinations, SDKs with foreign default endpoints (OpenAI, Anthropic, Gemini, Hugging Face Inference, `gradio.load("models/…")`), local serving (llama.cpp, vLLM, Ollama, Transformers, in-browser WebGPU), frameworks with telemetry on by default and their off-switches (Streamlit, Gradio, Chroma, vLLM), trackers, opaque bundles, licence, container, lockfiles, tests and offline tests.
- **A public registry** of 123 destinations with operator, jurisdiction (local, CH, EU, US…) and category — editable by pull request.
- **A deterministic grader** implementing the three rules; every reason cites evidence IDs.
- **Apertus v1.5 8B** (Q4 GGUF) served locally by **llama.cpp** on 4 CPU cores, called through an OpenAI-compatible client with **JSON-schema structured outputs**, temperature 0, a fixed seed and a response cache — so reports are reproducible. Any Apertus endpoint (e.g. 70B on CSCS) works via `LLM_BASE_URL`.
- **A web interface** that adopts America.gov's design system — display serif, tight sans, 48 px hero shell, pill controls, manifesto paragraph with inline icons, oversized footer wordmark — in Swiss colours (flag red RGB 255/0/0 for the cross, Pantone 485 C on screen for actions). Fonts and icons are bundled, strict CSP, no third-party request, no visitor logs.
- **Docker**: `make run` (Sigillum + local Apertus), `make offline-test` (18 tests + every sample audit inside a container with `network_mode: none`).

Challenges we ran into

- **False positives.** Our first community run counted documentation hyperlinks as data flows and missed licences declared in Space metadata. We inspected every finding by hand before publishing anything, fixed both, and added tests. Rigor first: a wrong seal is worse than no seal.
- **Measuring ourselves.** Sigillum's own detection tables (lists of SDK and host names) triggered its own detectors. We excluded them in a `.sigillumignore` that is *published in every report*, and made sure exclusions can hide flows but never hide tests.
- **A model that sees contradictions everywhere.** On a project graded A, Apertus 8B "contradicted" the claim *runs fully offline* by citing the local llama.cpp server as evidence. We now show Apertus only the traces that *can* contradict a claim (foreign flows, telemetry, downloads, opaque files) and reject any gap that cites anything else — and we log such cases as feedback for the Apertus team.
- **A small model on a CPU.** Apertus 8B quantised sometimes answered in English when asked for German or French, or copied French into Romansh. Our validation layer caught it every time and fell back to reviewed templates — and the report says so. That is exactly why the model explains but never grades.
- **A few hours.** We switched to this idea late and kept the scope tight: five questions, one scale, one report — end to end.

Accomplishments that we're proud of

- A working tool end to end: CLI, web UI, JSON reports, badges, `make run`, offline proof in Docker.
- A seal that is honest about itself: "unknown" is a first-class answer, rejections are counted, limits are published, and we graded ourselves before anyone else.
- A real, anonymous snapshot of the public Apertus ecosystem in October 2026 — a baseline the organisers can compare across the on-site hacks.

What we learned

- Most sovereignty failures are not dramatic: a default telemetry flag, a CDN script, a convenience SDK. They are cheap to fix once someone points at the exact line.
- Language models are most useful where rules fail — reading prose and explaining it — and most trustworthy when every answer is checked against evidence.
- Saying "unknown" builds more trust than a confident guess.

What's next for Sigillum

- A runtime probe that records outbound connections during a project's own tests, turning "inferred" answers into reproducible ones.
- A self-service flow for Hack Apertus teams: request a seal, read the report, accept or contest it before anything is published.
- Explanations on Apertus 70B, with native-speaker review of the Romansh and Italian templates.
- Offering the seal to the organisers for the on-site hacks and the May 2027 Grand Finals, and to public partners as a procurement aid.

> Sovereignty is not declared. It is proven. Apertus is open so that trust can be verified; Sigillum makes it readable.

Additional info — for judges and organisers (not public)

- **Track:** 2B — Own Project. Submitted through hackapertus.ch/online-hack/submissions as required; this Devpost page mirrors it.
- **Repository:** follows the HackApertus project template; project root `track_2b/` (README, technical_report.md, `Sigillum_Report.pdf`, Makefile, src/, data/, docs/).
- **Run:** `make run` from `track_2b/`. If `LLM_BASE_URL` / `LLM_API_KEY` / `LLM_NAME` are set (e.g. your CSCS Apertus endpoint), Sigillum uses them; otherwise it downloads Apertus v1.5 8B GGUF (~5 GB, build time) and serves it locally with llama.cpp. `make run-lite` runs without any model (deterministic grades, template explanations). `make test` and `make offline-test` reproduce the tests.
- **Target architecture:** (b) air-gapped, therefore also (a) on-premise; (c) Swiss cloud possible by setting `LLM_BASE_URL`. Zero runtime dependencies (Python stdlib).
- **Use of Apertus:** claims extraction, claim-vs-trace reconciliation, multilingual plain-language explanations — all validated against evidence; Apertus never sets the grade. No other model was used.
- **Ethics:** only public code was examined; the community audit is aggregated and anonymous (the aggregation code cannot output names); no Hack Apertus submission was examined. We would be glad to offer a free seal to any team that asks, with the right of reply before anything is published.
- **Honest limits:** static analysis only; hand-curated registry; Apertus 8B on CPU is conservative in its explanations (Romansh to be proof-read). Full list in `docs/method.md`.
- **Files:** `Sigillum_Submission.zip` — technical report (PDF), submission summary (PDF), pitch deck (PDF + 3:2 PNG slides), demo video (MP4).

Upload a file

`Sigillum_Submission.zip` (< 35 MB): `Sigillum_Report.pdf`, `Sigillum_Submission.pdf`, `Sigillum_Deck.pdf`, `slides/*.png` (3:2), `sigillum-demo.mp4`.